

FIT3155: Week 6 Lab Sheet

(Questions are based on Week 5 topic dealing with lossless data compression.)

Objectives: Develop a working understanding of Huffman coding for characters and Elias omega coding for integers, and apply these concepts in the context of LZ77 and LZSS lossless text compression algorithms.

Conceptual exercises

1. Let the text $\text{txt}[1 \dots M]$ contain n distinct symbols $\{s_1, s_2, \dots, s_n\}$ with corresponding frequencies $\{f_1, f_2, \dots, f_n\}$, ordered such that $f_1 \leq f_2 \leq \dots \leq f_n$. Let $\mathcal{H} = \{cw_1, cw_2, \dots, cw_n\}$ denote the corresponding binary Huffman codewords with lengths $\{l_1, l_2, \dots, l_n\}$.

- (a) Express the total number of bits required to encode the text using the code $\mathcal{H}$ in terms of $\{f_i\}$ and $\{l_i\}$.

Ans: 1(a)

Each distinct symbol s_i has an Huffman codeword length of l_i and appears f_i times in $\text{txt}[1 \dots M]$. Therefore, symbol s_i 's encoding length across all its occurrences in the text amounts to $f_i l_i$ bits. Summing this up over all distinct symbols, the total number of bits to encode the entire text requires $\sum_{i=1}^n f_i l_i$ bits.

- (b) Considering all possibilities of positive symbol frequencies of $n \geq 2$ symbols, what are the minimum and maximum possible values that any codeword length l_i can take?

Ans: 1(b)

The minimum possible value of any l_i is 1, as this translates to the minimum possible depth of any leaf node in a strict binary tree (with $n \geq 2$ leaves), which arises when the leaf node representing some s_i is a direct child of the root node.

The maximum possible value of l_i is $n - 1$, as this again translates to the maximum possible depth of any leaf node which arises in the most imbalanced version of a strict binary tree with n leaves, where each of the $n - 1$ internal nodes has exactly one leaf node as a direct child.

So, in general, $1 \leq l_i \leq n - 1$.

- (c) Is it possible for a Huffman code tree to have a height less than $\lceil \log_2 n \rceil$ bits? If yes, why? If no, why not?

Ans: 1(d)

No.

Consider a strict binary tree with n leaves. We first reason about the minimum possible height* $h_{\min}$ of such a tree. To achieve the minimum height, the tree must be as balanced

*Height of a binary tree is the longest path length from the root to any leaf in that tree.

as any strict binary tree with n leaves will allow. A strict binary tree is most balanced when it is a (near) complete binary tree[†], with all levels fully filled except possibly the last. In a (near) complete binary tree of height $h_{\min}$, number of leaves n 's relation with tree height $h_{\min}$ satisfies:

$$\begin{aligned} 2^{h_{\min}-1} &< n \leq 2^{h_{\min}} \\ \implies h_{\min} - 1 &< \log_2(n) \leq h_{\min} \\ \implies h_{\min} &= \lceil \log_2(n) \rceil \quad (\because h_{\min} \text{ is an integer}) \end{aligned}$$

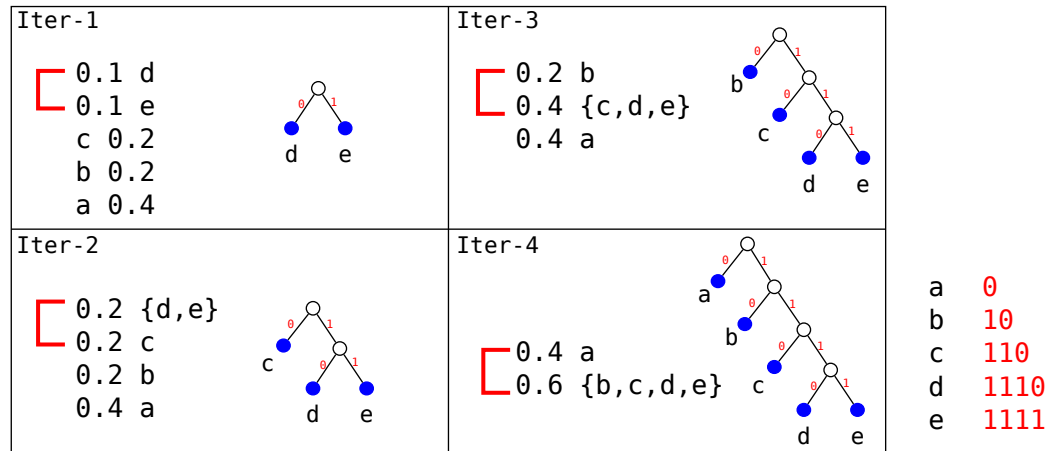
Therefore, for any Huffman coding tree with n leaves, this gives the lower bound for its height h :

$$h \geq h_{\min} = \lceil \log_2(n) \rceil.$$

2. Construct on paper a Huffman code for the set of symbols $\{a, b, c, d, e\}$ with corresponding probabilities $\{0.4, 0.2, 0.2, 0.1, 0.1\}$.

Ans: 2

Note that multiple alternative Huffman code trees can exist for a given set of symbol probabilities, depending on how ties are resolved during construction. In other words, the final tree structure depends much on which pair of lowest-probability nodes are greedily selected for merging when multiple choices are available. Below is one of several possible trees, all of which are optimal in the sense that they yield the same average codeword length.



Additional insights (for consideration but not examinable): It will be useful to work out the (probability) weighted mean and variance of the Huffman codeword lengths. These are worked out shown below.

Expected value (or mean) of the codeword lengths:

$$\begin{aligned} \mathbb{E}[L] &= \sum_{i=1}^n p_i l_i \\ &= 0.4 \times 1 + 0.2 \times 2 + 0.2 \times 3 + 0.1 \times 4 + 0.1 \times 4 \\ &= 0.4 + 0.4 + 0.6 + 0.4 + 0.4 \\ &= 2.2 \text{ bits.} \end{aligned}$$

Note that all alternative Huffman code trees for a given set of symbol probabilities achieve the same expected codeword length, which is the optimal (minimum) value.

Variance of the codeword lengths:

$$\begin{aligned} \text{Var}(L) &= \sum_{i=1}^n p_i (l_i - \mathbb{E}[L])^2 \\ &= 0.4(1 - 2.2)^2 + 0.2(2 - 2.2)^2 + 0.2(3 - 2.2)^2 + 0.1(4 - 2.2)^2 + 0.1(4 - 2.2)^2 \\ &= 0.4 \times 1.44 + 0.2 \times 0.04 + 0.2 \times 0.64 + 0.1 \times 3.24 + 0.1 \times 3.24 \\ &= 0.576 + 0.008 + 0.128 + 0.324 + 0.324 \\ &= 1.36. \end{aligned}$$

[†]This is the same kind of tree structure you encountered in the binary heap data structure

Note that alternative Huffman code trees can vary in their computed codeword length variance, even though they all will/should have the same expected codeword length. Variance of codeword lengths depends on how ties were resolved during construction. Certain additional rules around tie-breaking (as in the next question) allows one to minimize the variance (beyond just the mean).

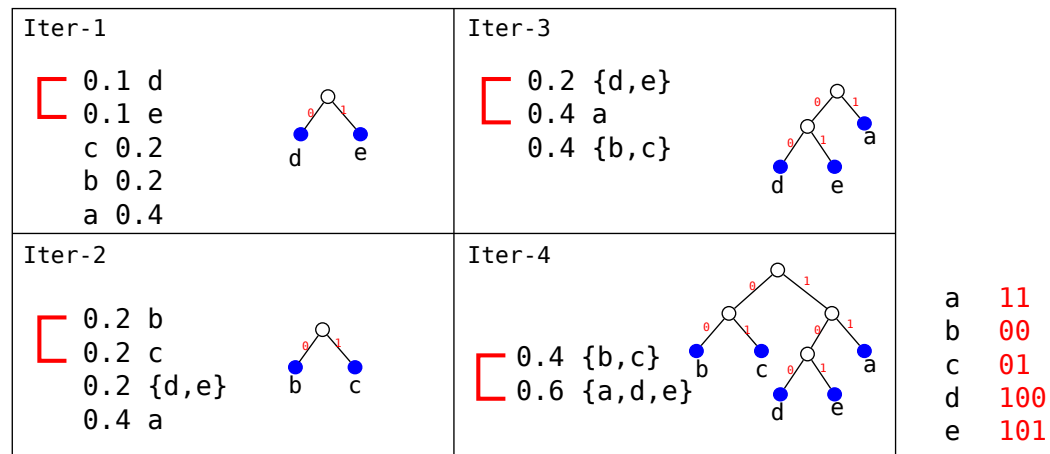
3. In the exercise above, you may have observed that when merging two nodes (or subtrees) with the smallest probabilities, there can be multiple valid choices, and hence select any pair (with smallest probabilities) among the choices.

A **variation** of Huffman coding algorithm resolves such ties by *always* merging the two subtrees with the smallest heights (among those with minimum probabilities). This yields what is termed as *minimum-variance* Huffman code.

Construct on paper a minimum-variance Huffman code for the same set of symbols and probabilities. Compare the resulting codewords with those obtained from the standard Huffman coding.

Ans: 3

Following the prescription, the resultant tree is:



Additional insights (for consideration but not examinable): Recomputing the weighted mean and variance as before, we have the following.

Expected value (or mean) of the codeword lengths:

$$\begin{aligned}
 \mathbb{E}[L] &= \sum_{i=1}^n p_i l_i \\
 &= 0.4 \times 2 + 0.2 \times 2 + 0.2 \times 2 + 0.1 \times 3 + 0.1 \times 3 \\
 &= 0.8 + 0.4 + 0.4 + 0.3 + 0.3 \\
 &= 2.2 \text{ bits.}
 \end{aligned}$$

Variance of the codeword lengths:

$$\begin{aligned}
 \text{Var}(L) &= \sum_{i=1}^n p_i (l_i - \mathbb{E}[L])^2 \\
 &= 0.4(2 - 2.2)^2 + 0.2(2 - 2.2)^2 + 0.2(2 - 2.2)^2 + 0.1(3 - 2.2)^2 + 0.1(3 - 2.2)^2 \\
 &= 0.4 \times 0.04 + 0.2 \times 0.04 + 0.2 \times 0.04 + 0.1 \times 0.64 + 0.1 \times 0.64 \\
 &= 0.016 + 0.008 + 0.008 + 0.064 + 0.064 \\
 &= 0.16.
 \end{aligned}$$

Notice that we obtained the same mean but a lower variance. By using a tie-breaking strategy that merges subtrees with smaller heights when multiple choices exist, one can obtain a Huffman tree

that minimizes the variance among all optimal Huffman trees. The earlier construction guarantees optimal average length but not minimal variance.

4. Does Huffman encoding always yield prefix-free codewords for the symbols it encodes? If yes, explain why. If not, provide a counterexample.

Ans: 4

Yes, codewords from an Huffman code tree are always prefix-free by construction, because code-word defines a binary (left/right) path from the root to a specific leaf. Since no leaf node is an ancestor of another leaf node, no codeword can be a prefix of another codeword.

Note: For completeness on the topic of Huffman codes, a **non-examinable** proof of the optimality of Huffman coding is appended at the end of this solutions sheet.

5. Construct the Elias omega codewords for 65535 and 65536. Then, decode each of the constructed codewords.

Ans: 5

Encoding $n = 65535$.

Code component: The minimal binary code for 65535 is

^{15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0}
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

First length component: The first length component encodes the minimal binary code of $16 -$

$1 = 15$, which is ^{3 2 1 0}1 1 1 1. Since this is a length component, we flag it by unsetting the most significant bit, which is always 1 in any minimal binary representation: ^{3 2 1 0}0 1 1 1.

Second length component: The second length component encodes the minimal binary code of

$4 - 1 = 3$, which is ^{1 0}1 1. Again, as a length component, we unset the most significant bit: ^{1 0}0 1.

Third length component: The third length component encodes the minimal binary code of $2 -$

$1 = 1$, which is ⁰1. Unsetting the most significant bit gives: ⁰0.

Combining all components: Combining all components gives the Elias omega code for

^{22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0}
 $n = 65535$: 0 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

^{22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0}
Decoding: 0 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

- Start with the first bit (reading bits left to right).
- It is ⁰0, so it represents a length component. Set 0 to 1, giving ⁰1 and read the next two bits.
- The next two bits are ^{1 0}0 1. Since it starts with a 0, it is again a length component. Set the most significant 0 to 1, giving ^{1 0}1 1. This is 3 in decimal. This implies the next component is 4 bits long, so read the next 4 bits.
- The next four bits are ^{3 2 1 0}0 1 1 1. Since it starts with a 0, it is another length component. Set the most significant 0 to 1, giving ^{3 2 1 0}1 1 1 1. This is 15 in decimal. This implies the next component is 16 bits long, so read the next 16 bits.
- The next 16 bits are ^{15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0}1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1. Since it starts with a 1, this is a code component. Decode the bits to $n = 65535$. Stop.

Encoding $n = 65536$.

Code component: The minimal binary code for 65536 is

^{16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0}
1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0.

First length component: The first length component encodes the minimal binary code of $17 - 1 = 16$, which is $\overset{4}{1} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0}$. Since this is a length component, we flag it by unsetting the most significant bit, which is always 1 in any minimal binary representation: $\overset{4}{0} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0}$.

Second length component: The second length component encodes the minimal binary code of $5 - 1 = 4$, which is $\overset{2}{1} \overset{1}{0} \overset{0}{0}$. Again, as a length component, we unset the most significant bit: $\overset{2}{0} \overset{1}{0} \overset{0}{0}$.

Third length component: The third length component encodes the minimal binary code of $3 - 1 = 2$, which is $\overset{1}{1} \overset{0}{0}$. Unsetting the most significant bit gives: $\overset{1}{0} \overset{0}{0}$.

Fourth length component: The fourth length component encodes the minimal binary code of $2 - 1 = 1$, which is $\overset{0}{1}$. Unsetting the most significant bit gives: $\overset{0}{0}$.

Combining all components: Combining all components gives the Elias omega code for $n = 65536$: $\overset{27}{0} \overset{26}{0} \overset{25}{0} \overset{24}{0} \overset{23}{0} \overset{22}{0} \overset{21}{0} \overset{20}{0} \overset{19}{0} \overset{18}{0} \overset{17}{0} \overset{16}{0} \overset{15}{0} \overset{14}{0} \overset{13}{0} \overset{12}{0} \overset{11}{0} \overset{10}{0} \overset{9}{0} \overset{8}{0} \overset{7}{0} \overset{6}{0} \overset{5}{0} \overset{4}{0} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0}$.

Decoding: $\overset{27}{0} \overset{26}{0} \overset{25}{0} \overset{24}{0} \overset{23}{0} \overset{22}{0} \overset{21}{0} \overset{20}{0} \overset{19}{0} \overset{18}{0} \overset{17}{0} \overset{16}{0} \overset{15}{0} \overset{14}{0} \overset{13}{0} \overset{12}{0} \overset{11}{0} \overset{10}{0} \overset{9}{0} \overset{8}{0} \overset{7}{0} \overset{6}{0} \overset{5}{0} \overset{4}{0} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0}$.

- Start with the first bit (left to right).
- It is $\overset{0}{0}$, so it represents a length component. Set 0 to 1, giving $\overset{0}{1}$ and read the next two bits.
- The next two bits are $\overset{1}{0} \overset{0}{0}$. Since it starts with a 0, it is again a length component. Set the most significant 0 to 1, giving $\overset{1}{1} \overset{0}{0}$. This is 2 in decimal. This implies the next component is 3 bits long, so read the next 3 bits.
- The next 3 bits are $\overset{2}{0} \overset{1}{0} \overset{0}{0}$. Since it starts with a 0, it is another length component. Set the most significant 0 to 1, giving $\overset{2}{1} \overset{1}{0} \overset{0}{0}$. This is 4 in decimal. This implies the next component is 5 bits long, so read the next 5 bits.
- The next 5 bits are $\overset{4}{0} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0}$. Since it starts with a 0, it is another length component. Set the most significant 0 to 1, giving $\overset{4}{1} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0}$. This is 16 in decimal. This implies the next component is 17 bits long, so read the next 17 bits.
- The next 17 bits are $\overset{16}{1} \overset{15}{0} \overset{14}{0} \overset{13}{0} \overset{12}{0} \overset{11}{0} \overset{10}{0} \overset{9}{0} \overset{8}{0} \overset{7}{0} \overset{6}{0} \overset{5}{0} \overset{4}{0} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0}$. Since it starts with a 1, this is a code component. Decode it to $n = 65536$. Stop.

6. The Elias omega code constructs uniquely decodable codes for any $N \in \mathbb{Z}^+$. For certain values of N , the number of length components increases by one compared to those required to encode the previous number $N - 1$. Is there a general pattern governing when this increase occurs?

Ans: 6

Consider a number $N \geq 1$ whose Elias omega code contains $k \geq 1$ components (including both length and code components). Label these components (from left to right) as $\{c_1, c_2, \dots, c_k\}$ (Note: these components are generated in the opposite order $c_k, c_{k-1}, \dots, c_1$ during encoding.)

Analyze the relationship between the lengths of each component c_i (denoted here by $|c_i|$), starting from the code component c_k .

The code component c_k contains exactly $\lfloor \log_2 N \rfloor + 1$ bits. For convenience, this is approximately $\log_2 N$ bits.

This implies that c_{k-1} contains exactly $\lfloor \log_2(|c_k| - 1) \rfloor + 1$ bits, which is $\approx \log_2(|c_k| - 1) \approx \log_2 \log_2 N$ bits.

Similarly, c_{k-2} contains approximately $\log_2 \log_2 \log_2 N$ bits, and so on.

Thus, the lengths of each component during encoding follow an iterated logarithm ($\log^*$) pattern of the form

$$\log N, \log \log N, \log \log \log N, \dots, 1.$$

Therefore, the number of components k is determined by the number of times $\log_2$ is applied to N before the binary encoding reduces to $|c_1| = 1$ bit.

Now, the number of components increases by one precisely when an additional iteration of the logarithm is required. This occurs at specific values of N , as we begin to invert the iterated

logarithm starting from c_1 , which is 1 bit long. We will use Knuth's double-arrow notation[‡] below:

Starting from $|c_1| = 1$, observe how components can grow in their limits.

- $|c_1| = 1$. This means $|c_2| = 2 = 2^1 = 2 \uparrow\uparrow 1$.
- If c_2 encodes the largest 2-bit number, which is $3 = 2^2 - 1$, then we have $|c_3| = 4 = 2^2 = 2 \uparrow\uparrow 2$.
- If c_3 encodes the largest 4-bit number, which is $15 = 2^{2^2} - 1$, then we have $|c_4| = 16 = 2^{2^2} = 2 \uparrow\uparrow 3$.
- If c_4 encodes the largest 16-bit number, which is $65535 = 2^{2^{2^2}} - 1$ then we have $|c_5| = 65536 = 2^{2^{2^2}} = 2 \uparrow\uparrow 4$.
- $\vdots$ (and so on)

As we saw above, this sequence corresponds to repeated exponentiation of 2. Hence, an additional component is introduced exactly when any N reaches values of the form: $N = 2, 2^2, 2^{2^2}, 2^{2^{2^2}}, \dots$ or equivalently, $N = 2 \uparrow\uparrow k$ for $k \geq 1$.

At these values, one additional application of $\log_2$ is required (compared to $N - 1$) to reduce the value down to 1. (We saw such a transition in the previous question where $N = 65535$ had 4 components over all in the encoding, while the next number $N = 65536$ had 5 components.)

In summary, the number of components increases by one exactly at whenever a number transitions from $N = (2 \uparrow\uparrow k) - 1 \rightarrow 2 \uparrow\uparrow k$ for all $k \geq 1$.

7. Encode the following sequence into $\langle \text{offset}, \text{length}, \text{character} \rangle$ triples using the LZ77 algorithm:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
b a r r a y a r _ b a r _ b y _ b a r r a y a r _ b a y

Assume that the search window size and lookahead buffer size are both 15.

Ans: 7

Below, search window is in **blue** (max size 15), lookahead buffer is in **violet** (max size 15), and substring-copy (where applicable) is in **red**.

Iteration	State	LZ77 tuple
1	b a r r a y a r _ b a r _ b y _ b a r r a y a r _ b a y	$\langle 0, 0, b \rangle$
2	b a r r a y a r _ b a r _ b y _ b a r r a y a r _ b a y	$\langle 0, 0, a \rangle$
3	b a r r a y a r _ b a r _ b y _ b a r r a y a r _ b a y	$\langle 0, 0, r \rangle$
4	b a r r a y a r _ b a r _ b y _ b a r r a y a r _ b a y	$\langle 1, 1, a \rangle$
5	b a r r a y a r _ b a r _ b y _ b a r r a y a r _ b a y	$\langle 0, 0, y \rangle$
6	b a r r a y a r _ b a r _ b y _ b a r r a y a r _ b a y	$\langle 5, 2, - \rangle$
7	b a r r a y a r _ b a r _ b y _ b a r r a y a r _ b a y	$\langle 9, 3, - \rangle$
8	b a r r a y a r _ b a r _ b y _ b a r r a y a r _ b a y	$\langle 4, 1, y \rangle$
9	b a r r a y a r _ b a r _ b y _ b a r r a y a r _ b a y	$\langle 7, 4, r \rangle$
10	b a r r a y a r _ b a r _ b y _ b a r r a y a r _ b a y	$\langle 3, 1, y \rangle$
11	b a r r a y a r _ b a r _ b y _ b a r r a y a r _ b a y	$\langle 12, 4, a \rangle$
12	b a r r a y a r _ b a r _ b y _ b a r r a y a r _ b a y	$\langle 6, 1, \$ \rangle$

Note: In the last iteration (#12), the lookahead buffer contains only one character, and the algorithm finds a match in the search window of length 1. Consequently, there is technically no `next_char` available to include in the tuple. One conservative way to handle this is via a new unique terminal symbol (say `$`, as done in iteration #12). However, this is not the only approach. More practically, when the match extends to the natural end of the input string, the copy length can instead be reduced by 1 (so that it excludes the final character), and that character can then be encoded as the `next_char`. In the extreme case where the copy length is 1 (as above) in the last iteration, the tuple can be encoded as $\langle 0, 0, \text{last_char} \rangle$ (or $\langle 0, 0, y \rangle$ in the above example).

[‡]Knuth's double-arrow notation provides a compact representation of power towers (tetration) of 2. For example, $2^{2^{2^2}} = 2 \uparrow\uparrow 4$.

8. Decode the encoded triples from the previous part to recover the original text.

Ans: 8

$\langle 0, 0, b \rangle \rightarrow b$
 $\langle 0, 0, a \rangle \rightarrow ba$
 $\langle 0, 0, r \rangle \rightarrow bar$
 $\langle 1, 1, a \rangle \rightarrow barra$
 $\langle 0, 0, y \rangle \rightarrow barray$
 $\langle 5, 2, - \rangle \rightarrow barrayar-$
 $\langle 9, 3, - \rangle \rightarrow barrayar.-$
 $\langle 4, 1, y \rangle \rightarrow barrayar.bar.by$
 $\langle 7, 4, r \rangle \rightarrow barrayar.bar.by.barr$
 $\langle 3, 1, y \rangle \rightarrow barrayar.bar.by.barray$
 $\langle 12, 4, a \rangle \rightarrow barrayar.bar.by.barrayar.ba$
 $\langle 0, 0, y \rangle \rightarrow barrayar.bar.by.barrayar.bay$

9. Repeat the previous two questions using the same input string, but apply the LZSS variant of the algorithm. To vary things a bit, increase the search window and look ahead buffer size to infinity.

Ans: 9

Encoding

Below, search window is in **blue** (max size infinity), lookahead buffer is in **violet** (max size infinity), and substring-copy (where applicable) is in **red**.

Iteration	State	LZSS tuple
1	barrayar.bar.by.barrayar.bay	$\langle 1, b \rangle$
2	b arrayar.bar.by.barrayar.bay	$\langle 1, a \rangle$
3	ba rrayar.bar.by.barrayar.bay	$\langle 1, r \rangle$
4	bar rayar.bar.by.barrayar.bay	$\langle 1, r \rangle$
5	barr ayar.bar.by.barrayar.bay	$\langle 1, a \rangle$
6	barra yar.bar.by.barrayar.bay	$\langle 1, y \rangle$
7	barray ar.bar.by.barrayar.bay	$\langle 1, a \rangle$
8	barraya r.bar.by.barrayar.bay	$\langle 1, r \rangle$
9	barrayar .bar.by.barrayar.bay	$\langle 1, - \rangle$
10	barrayar.-bar.by.barrayar.bay	$\langle 0, 9, 3 \rangle$
11	barrayar.bar_.by.barrayar.bay	$\langle 1, - \rangle$
12	barrayar.bar_.by.barrayar.bay	$\langle 1, b \rangle$
13	barrayar.bar.b_y.barrayar.bay	$\langle 1, y \rangle$
14	barrayar.bar.by_.barrayar.bay	$\langle 0, 7, 4 \rangle$
15	barrayar.bar.by.bar rayar.bay	$\langle 0, 16, 8 \rangle$
16	barrayar.bar.by.barrayar.ba y	$\langle 1, y \rangle$

Decoding

$\langle 1, b \rangle \rightarrow b$
 $\langle 1, a \rangle \rightarrow ba$
 $\langle 1, r \rangle \rightarrow bar$
 $\langle 1, r \rangle \rightarrow barr$
 $\langle 1, a \rangle \rightarrow barr a$
 $\langle 1, y \rangle \rightarrow barra y$
 $\langle 1, a \rangle \rightarrow barray a$
 $\langle 1, r \rangle \rightarrow barraya r$
 $\langle 1, - \rangle \rightarrow barrayar -$
 $\langle 0, 9, 3 \rangle \rightarrow barrayar_bar$
 $\langle 1, - \rangle \rightarrow barrayar_bar -$
 $\langle 1, b \rangle \rightarrow barrayar_bar_b$
 $\langle 1, y \rangle \rightarrow barrayar_bar_by$
 $\langle 0, 7, 4 \rangle \rightarrow barrayar_bar_by_bar$
 $\langle 0, 16, 8 \rangle \rightarrow barrayar_bar_by_bar_rayar_ba$
 $\langle 1, y \rangle \rightarrow barrayar_bar_by_barrayar_ba y$

10. A text is encoded using the LZ77 algorithm, yielding the following sequence of triples:

$\langle 0, 0, r \rangle$, $\langle 0, 0, a \rangle$, $\langle 0, 0, t \rangle$, $\langle 2, 8, - \rangle$, $\langle 3, 1, - \rangle$, $\langle 0, 0, r \rangle$, $\langle 6, 4, t \rangle$, $\langle 9, 5, t \rangle$

Decode the sequence to recover the original text.

Ans: 10

$\langle 0, 0, r \rangle \rightarrow r$
 $\langle 0, 0, a \rangle \rightarrow ra$
 $\langle 0, 0, t \rangle \rightarrow rat$
 $\langle 2, 8, - \rangle \rightarrow ratatatatat_$
 $\langle 3, 1, - \rangle \rightarrow ratatatatat_a_$
 $\langle 0, 0, r \rangle \rightarrow ratatatatat_a_r$
 $\langle 6, 4, t \rangle \rightarrow ratatatatat_a_rat_at$
 $\langle 9, 5, t \rangle \rightarrow ratatatatat_a_rat_at_a_rat$

Recovered string ^{1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26}
 r a t a t a t a t a t _ a _ r a t _ a t _ a _ r a t

-=0=-
 END
 -=0=-

Non-examinable optimality proof of Huffman algorithm

Let $\Psi_n = \{s_1, s_2, \dots, s_n\}$ be a set of n symbols with positive frequencies $\{f_{s_1}, f_{s_2}, \dots, f_{s_n}\}$ (not necessarily ordered, in this proof).

We will show that the Huffman code $\mathcal{H}$ for the set Ψ_n constructed using the Huffman's (greedy) algorithm is optimal, in the sense that it *minimizes* the total encoding length $L(\mathcal{H}(\Psi_n)) = \sum_{i=1}^n f_{s_i} l_{s_i}$.

Proposition: For any pair of (least-frequent) symbols $\{s_a, s_b\} \subseteq \Psi_n$ such that

$$f_{s_a} \leq f_{s_b} \leq \{f_{s_i}\}_{\forall i \neq a, b}$$

there exists an optimal Huffman code $\mathcal{H}^*(\Psi_n)$ with codeword lengths $\{l_{s_i}^*\}$ in which s_a and s_b share the same parent and are assigned the longest codeword lengths in that tree, $l_{s_a}^* = l_{s_b}^* \geq \{l_{s_i}^*\}_{\forall i \neq a, b}$.

Justification: Let us prove this by contradiction. Assume that there exists no optimal Huffman code $\mathcal{H}^*(\Psi_n)$ in which s_a and s_b are assigned the longest codeword lengths as defined above. Then there should exist symbols s_x and s_y ($s_x \neq s_a$ or $s_y \neq s_b$) that are assigned the longest codewords sharing the same parent in $\mathcal{H}^*(\Psi_n)$.

Now, consider a perturbed version of $\mathcal{H}^*(\Psi_n)$, denoted by $\overline{\mathcal{H}}^*(\Psi_n)$, obtained by swapping s_a with s_x , or s_b with s_y . Note that the Huffman tree defined by $\mathcal{H}^*(\Psi_n)$ is 'isomorphic' to the perturbed tree $\overline{\mathcal{H}}^*(\Psi_n)$, meaning that they have the same strict binary tree structure but differ only in the assignment of symbols to leaves due to the swap.

Since $f_{s_a} \leq f_{s_x}$ and $f_{s_b} \leq f_{s_y}$, such a swap assigns same or smaller frequency symbol s_a to same or larger codeword length $l_{s_x}^*$ to yield:

$$L(\overline{\mathcal{H}}^*(\Psi_n)) \leq L(\mathcal{H}^*(\Psi_n)).$$

But by definition of optimality of $\mathcal{H}^*(\Psi_n)$, it has to be that

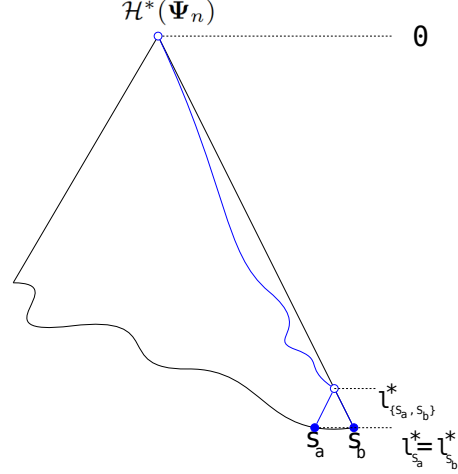
$$L(\mathcal{H}^*(\Psi_n)) \leq L(\overline{\mathcal{H}}^*(\Psi_n)).$$

The above two relations can be simultaneously true if and only if

$$L(\overline{\mathcal{H}}^*(\Psi_n)) = L(\mathcal{H}^*(\Psi_n))$$

thereby contradicting the assumption.

Hence, there exists an optimal Huffman code in which any pair of least-frequent symbols $\{s_a, s_b\}$ can be assigned the longest codeword lengths and share the same parent.



Subproblem decomposition

Consider an optimal code $\mathcal{H}^*$ over the set of n symbols Ψ_n as per the above proposition.

Merging s_a and s_b results in a new (merged) symbol $s_{\{s_a, s_b\}}$ with combined frequency of $f_{\{s_a, s_b\}} = f_{s_a} + f_{s_b}$. After the above merge, denote $\Psi_{n-1} = \Psi_n \setminus \{s_a, s_b\} \cup s_{\{s_a, s_b\}}$ to imply a reduced set of symbols of size $n-1$ with corresponding frequencies.

As illustrated in the figure above, since s_a and s_b have the same parent, it has to be that:

$$l_{s_a} = l_{\{s_a, s_b\}} + 1, \quad l_{s_b} = l_{\{s_a, s_b\}} + 1$$

Therefore, the total encoding length can be decomposed as follows:

$$\begin{aligned} L(\mathcal{H}^*(\Psi_n)) &= f_{s_a} l_{s_a}^* + f_{s_b} l_{s_b}^* + \sum_{s_i \neq s_a, s_b} f_{s_i} l_{s_i}^* \\ &= f_{s_a} (l_{\{s_a, s_b\}}^* + 1) + f_{s_b} (l_{\{s_a, s_b\}}^* + 1) + \sum_{s_i \neq s_a, s_b} f_{s_i} l_{s_i}^* \\ &= f_{s_a} + f_{s_b} + (f_{s_a} + f_{s_b}) l_{\{s_a, s_b\}}^* + \underbrace{\sum_{s_i \neq s_a, s_b} f_{s_i} l_{s_i}^*}_{\text{subproblem: } L(\mathcal{H}^*(\Psi_{n-1}))} \end{aligned}$$

Therefore:

$$L(\mathcal{H}^*(\Psi_n)) = f_{s_a} + f_{s_b} + L(\mathcal{H}^*(\Psi_{n-1})).$$

The term $f_{s_a} + f_{s_b}$ does not depend on the length of the codewords assigned in the subproblem. Hence, minimizing $L(\mathcal{H}^*(\Psi_n))$ implies minimizing $L(\mathcal{H}^*(\Psi_{n-1}))$. Same (inductive) reasoning applies for the subproblem.

Huffman's algorithm precisely gains from this decomposition by performing a greedy merge in each iteration ensuring the resultant tree

$$L(\mathcal{H}(\Psi_n)) = L(\mathcal{H}^*(\Psi_n)).$$